SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Mock Interview)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO3-AT3-MOCK INTERVIEW
Question	Q2 — Python Tuples vs. Lists (10 Marks)

Q2. [10 Marks] What is a tuple in Python? Why are tuples immutable? Give three real-world scenarios where a tuple is more appropriate than a list.

Answer

What is a tuple in Python?

A tuple is a built-in, ordered collection type in Python that, unlike a list, is **immutable** — once created, its contents cannot be changed. Tuples are written using parentheses, e.g. (1, 2, 3), and like lists they can hold elements of any data type, including mixed types, and support indexing, slicing, and iteration. A tuple can also be created without parentheses using just commas, and a single-element tuple requires a trailing comma, e.g. (5,).

Why are tuples immutable?

Tuples are designed to be immutable by the language itself: once a tuple object is created, no item inside it can be added, removed, or reassigned, and any attempt to do so raises a TypeError. This immutability is a deliberate design choice for several reasons:

- **Data integrity:** immutability guarantees that a tuple's contents cannot be accidentally or maliciously modified after creation, which is useful for representing fixed collections of related values.
- **Hashability:** because their contents cannot change, tuples (containing only hashable elements) can themselves be hashed. This lets them be used as dictionary keys or stored in sets, something a mutable list can never do.
- **Performance:** since the size and contents of a tuple are fixed at creation, Python can store and access tuples slightly more efficiently than lists, and they generally use less memory.
- **Predictability:** functions that accept a tuple as an argument can be certain the caller's data will not be modified as a side effect, making code easier to reason about.

Three real-world scenarios where a tuple is more appropriate than a list

- **Fixed geographic coordinates:** a location such as (latitude, longitude) represents a single, complete value whose two parts should never be changed independently — a tuple like (13.0827, 80.2707) is a natural fit, whereas a list might tempt code to overwrite just one coordinate and produce an invalid location.
- **Dictionary keys for composite data:** when data needs to be indexed by more than one value, for example (year, month) as a key into a sales dictionary, only a tuple can be used directly as a key because it is hashable; a list would raise a TypeError.

• **Returning multiple fixed values from a function:** a function that computes and returns several related results at once, such as (min_value, max_value, average), naturally returns a tuple, since the number and meaning of the returned values are fixed and should not be accidentally mutated by the caller.

Python Program

```
"""
CO3-AT3 - Mock Interview
Q2: Python Tuples vs Lists [10 Marks]
------------------------------------------------------------
Demonstrates tuple immutability, hashability as a dict key,
and typical real-world use cases compared with a list.
"""

# A tuple representing a fixed geographic coordinate
location = (13.0827, 80.2707)
print("Location tuple:", location)

# Attempting to modify a tuple raises a TypeError
try:
    location[0] = 20.0
except TypeError as e:
    print("Error modifying tuple:", e)

# Using a tuple as a dictionary key (composite key)
sales = {
    (2025, "Jan"): 45000,
    (2025, "Feb"): 52000,
}
print("Sales for Jan 2025:", sales[(2025, "Jan")])

# Returning multiple fixed values from a function as a tuple
def get_stats(numbers):
    return (min(numbers), max(numbers), sum(numbers) / len(numbers))

marks = [78, 92, 65, 88, 74]
min_val, max_val, avg_val = get_stats(marks)
print(f"Min: {min_val}, Max: {max_val}, Avg: {avg_val:.2f}")

# A list, by contrast, is mutable and cannot be used as a dict key
scores_list = [78, 92, 65]
scores_list[0] = 100
print("Modified list:", scores_list)
```

Sample Output

```
Location tuple: (13.0827, 80.2707)
Error modifying tuple: 'tuple' object does not support item assignment
Sales for Jan 2025: 45000
Min: 65, Max: 92, Avg: 79.40
Modified list: [100, 92, 65]
```

Conclusion

A Python tuple is an ordered, immutable collection that, once created, cannot be modified, added to, or shrunk. This immutability makes tuples hashable and therefore usable as dictionary keys or set members, gives them a slight performance and memory edge over lists, and protects fixed data such as coordinates from unintended change. As the program demonstrates, a tuple is the more appropriate choice whenever data represents a fixed,

related group of values that should not change — such as a coordinate pair, a composite dictionary key, or a set of values returned together from a function — whereas a list remains the right choice when the collection needs to grow, shrink, or be modified over time.